



jQuery



jQuery

- ❖ jQuery is a JavaScript Library.
- ❖ jQuery is a lightweight, "write less, do more", JavaScript library.
- ❖ The purpose of jQuery is to make it much easier to use JavaScript on your website.
- ❖ jQuery takes a lot of common tasks that require many lines of JavaScript code to accomplish and wraps them into methods that you can call with a single line of code.
- ❖ jQuery also simplifies a lot of the complicated things from JavaScript, like AJAX calls and DOM manipulation.



jQuery Library Features

- ▶ HTML/DOM manipulation
- ▶ CSS manipulation
- ▶ HTML event methods
- ▶ Effects and animations
- ▶ AJAX
- ▶ Utilities



Difference between Library and Framework

Difference between Library and Framework		
Basis	LIBRARY	FRAMEWORK
Meaning	Library is collection of reusable functions used by computer program	Framework is a piece of code that dictates the architecture of project
Inversion of control	With a library , you are in-charge, means you can choose where and when you want to insert or use the library.	In a framework, the framework is in-charge, not you, means a framework tells you where to put a specific part of your code
Function	They are important in program linking and binding process	In a framework , provided standard way to build and deploy applications
Flexibility	Libraries are more flexible with greater degree of control	Frameworks are enforced structure and standards.
Example	React.js, JQuery is a javascript library	Angular js, Vue js is javascript framework

jQuery on Your Website

- ▶ Download the jQuery library from [jQuery.com](http://jquery.com)
- ▶ Include jQuery from a CDN, like Google



jQuery Syntax

Basic syntax is: `$(selector).action()`

- ▶ A \$ sign to define/access jQuery
- ▶ A (selector) to "query (or find)" HTML elements
- ▶ A jQuery action() to be performed on the element(s)



Example

Syntax	Description
<code>\$("*")</code>	Selects all elements
<code>\$(this)</code>	Selects the current HTML element
<code>\$("#p.intro")</code>	Selects all <code><p></code> elements with <code>class="intro"</code>
<code>\$("#p:first")</code>	Selects the first <code><p></code> element
<code>\$("#ul li:first")</code>	Selects the first <code></code> element of the first <code></code>

Example

<code>\$("ul li:first-child")</code>	Selects the first <code></code> element of every <code></code>
<code>\$("[href]")</code>	Selects all elements with an href attribute
<code>\$("a[target='_blank']")</code>	Selects all <code><a></code> elements with a target attribute value equal to <code>"_blank"</code>
<code>\$("a[target!='_blank']")</code>	Selects all <code><a></code> elements with a target attribute value NOT equal to <code>"_blank"</code>
<code>\$(":button")</code>	Selects all <code><button></code> elements and <code><input></code> elements of type="button"
<code>\$("tr:even")</code>	Selects all even <code><tr></code> elements
<code>\$("tr:odd")</code>	Selects all odd <code><tr></code> elements



jQuery Events

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	Submit	load
dblclick	keydown	Change	resize
mouseenter	keyup	Focus	scroll
mouseleave		Blur	unload

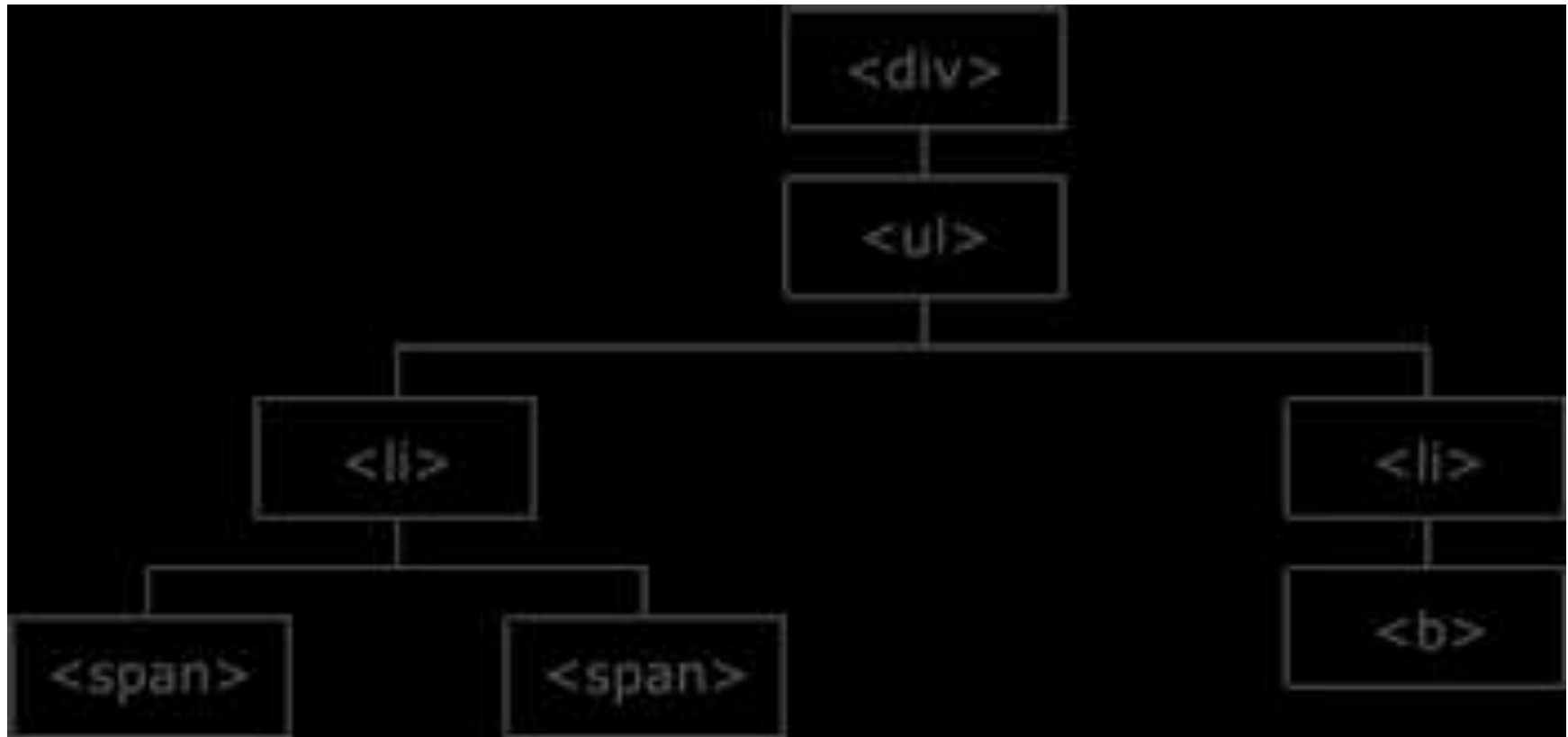


Traversing

- ▶ jQuery traversing, which means "move through", are used to "find" (or select) HTML elements based on their relation to other elements.
- ▶ Start with one selection and move through that selection until you reach the elements you desire.
- ▶ The image below illustrates an HTML page as a tree (DOM tree).
- ▶ With jQuery traversing, you can easily move up (ancestors), down (descendants) and sideways (siblings) in the tree, starting from the selected (current) element.
- ▶ This movement is called traversing - or moving through - the DOM tree.



DOM tree



jQuery Traversing - Ancestors

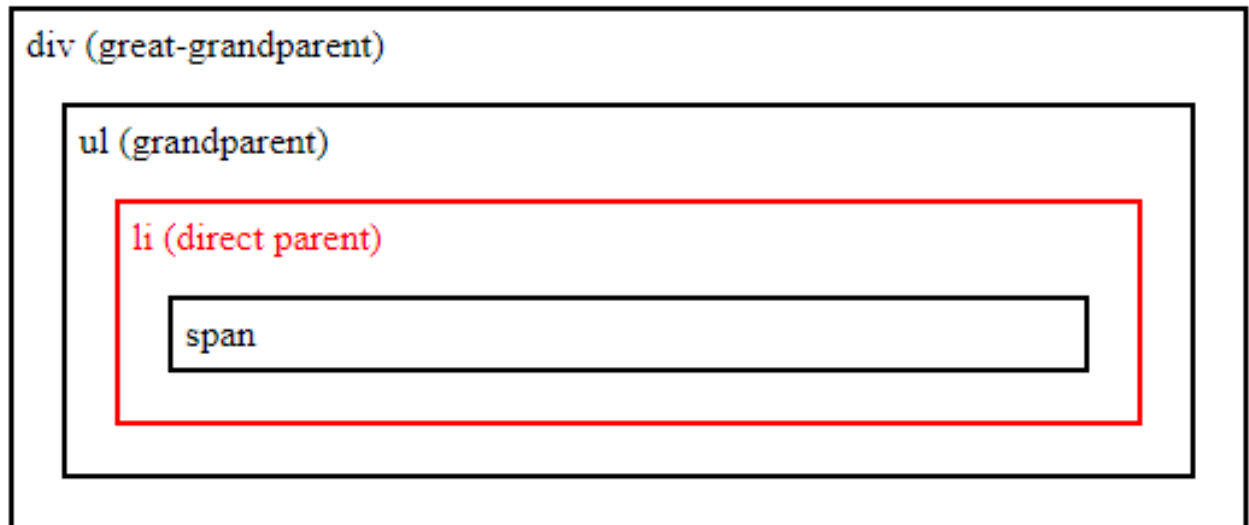
- ▶ Three useful jQuery methods for traversing up the DOM tree are:
 1. `parent()`
 2. `parents()`
 3. `parentsUntil()`



Parent()

- ▶ The `parent()` method returns the direct parent element of the selected element.
- ▶ This method only traverse a single level up the DOM tree.

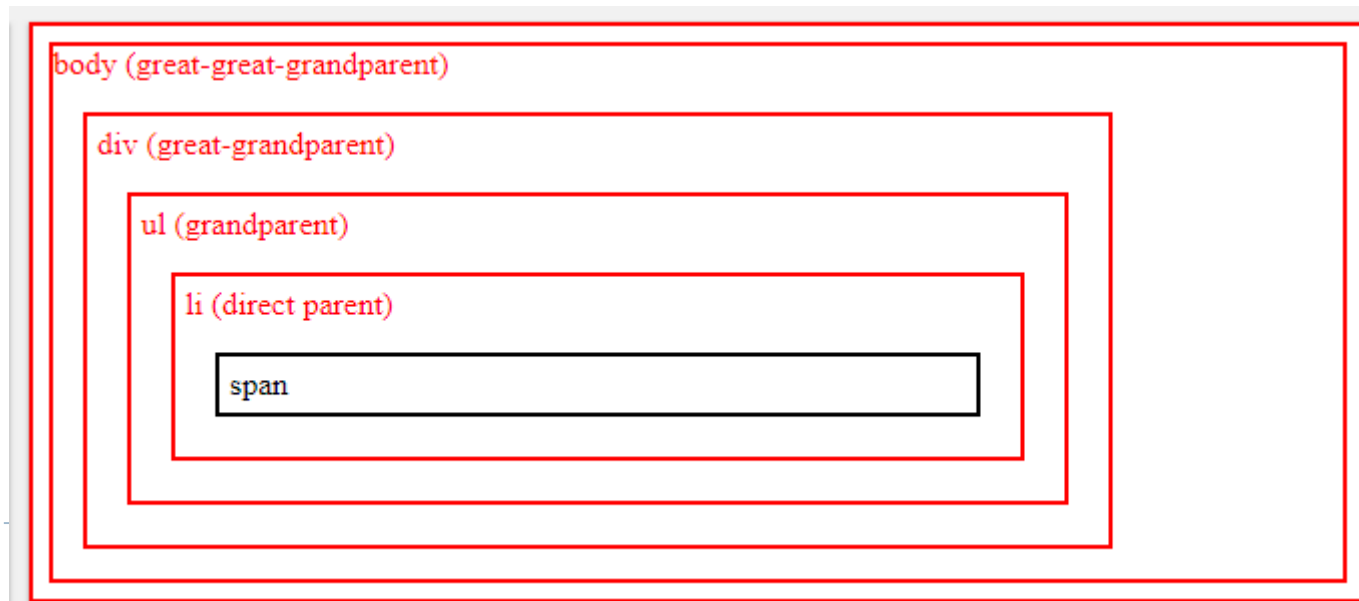
```
$(document).ready(function(){  
    $("span").parent().css({"color": "red", "border":  
    "2px solid red"})  
});
```



Parents()

- ▶ The `parents()` method returns all ancestor elements of the selected element, all the way up to the document's root element (`<html>`).

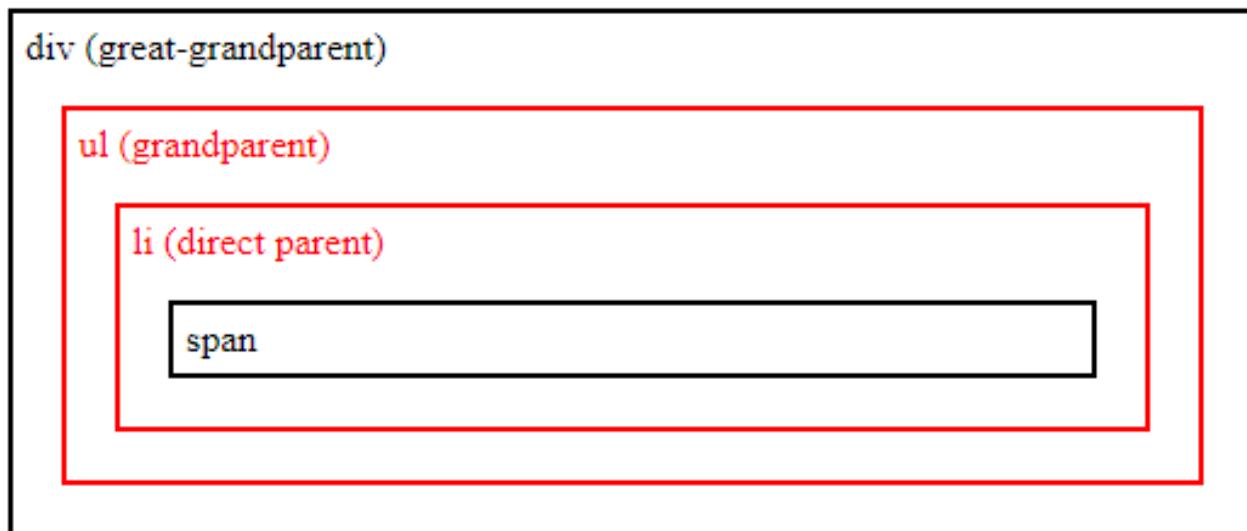
```
$(document).ready(function(){  
    $("span").parents().css({"color": "red", "border":  
    "2px solid red"});  
});
```



parentsUntil()

- ▶ The `parentsUntil()` method returns all ancestor elements between two given arguments.
- ▶ The following example returns all ancestor elements between a `<span>` and a `<div>` element:

```
$(document).ready(function(){  
    $("span").parentsUntil("div").css({"color": "red",  
    "border": "2px solid red"});  
});
```



jQuery Traversing - Decendants

- ▶ With jQuery you can traverse down the DOM tree to find descendants of an element.
- ▶ Two useful jQuery methods for traversing down the DOM tree are:
 1. `children()`
 2. `find()`



Children()

- ▶ The children() method returns all direct children of the selected element.
- ▶ This method only traverses a single level down the DOM tree.

```
$(document).ready(function(){  
    $("div").children().css({"color": "red", "border":  
    "2px solid red"});  
});
```

p (child)

span (grandchild)



find()

- ▶ The find() method returns descendant elements of the selected element, all the way down to the last descendant.

```
$(document).ready(function(){  
    $("div").find("span").css({"color": "red", "border":  
    "2px solid red"});  
});
```

p (child)

span (grandchild)



jQuery Traversing - Siblings

- ▶ With jQuery you can traverse sideways in the DOM tree to find siblings of an element.
- ▶ Siblings share the same parent.
- ▶ There are many useful jQuery methods for traversing sideways in the DOM tree.
- ▶ `siblings()`
- ▶ `next()`
- ▶ `nextAll()`
- ▶ `nextUntil()`
- ▶ `prev()`
- ▶ `prevAll()`
- ▶ `prevUntil()`



Siblings()

- ▶ The `siblings()` method returns all sibling elements of the selected element.

```
$(document).ready(function(){  
    $("h2").siblings();  
});
```



next()

- ▶ The next() method returns the next sibling element of the selected element.

```
$(document).ready(function(){  
    $("h2").next();  
});
```



nextAll()

- ▶ The `nextAll()` method returns all next sibling elements of the selected element.

```
$(document).ready(function(){  
    $("h2").nextAll();  
});
```



nextUntil()

- ▶ The nextUntil() method returns all next sibling elements between two given arguments.

```
$(document).ready(function(){  
    $("h2").nextUntil("h6");  
});
```



jQuery Traversing- Filtering

- ▶ The most basic filtering methods are `first()`, `last()` and `eq()`, which allow you to select a specific element based on its position in a group of elements.
- ▶ Other filtering methods, like `filter()` and `not()` allow you to select elements that match, or do not match, a certain criteria.



first()

The `first()` method returns the first element of the specified elements.

```
$(document).ready(function(){  
    $("div").first().css("background-color", "yellow");  
});
```



last()

- ▶ The last() method returns the last element of the specified elements.

```
$(document).ready(function(){  
    $("div").last().css("background-color", "yellow");  
});
```



filter()

- ▶ The filter() method lets you specify a criteria. Elements that do not match the criteria are removed from the selection, and those that match will be returned.
- ▶ The following example returns all <p> elements with class name "intro":

```
$(document).ready(function(){  
    $("p").filter(".intro").css("background-color",  
    "yellow");  
});
```



AJAX

- ▶ AJAX = Asynchronous JavaScript and XML.
- ▶ In short; AJAX is about loading data in the background and display it on the webpage, without reloading the whole page.
- ▶ Examples of applications using AJAX: Gmail, Google Maps, Youtube, and Facebook tabs.



load() Method

- ▶ The jQuery load() method is a simple, but powerful AJAX method.
- ▶ The load() method loads data from a server and puts the returned data into the selected element.

Syntax:

`$(selector).load(URL,data,callback);`



Example

```
$(document).ready(function(){  
    $("button").click(function(){  
        $("#div1").load("demo_test.txt");  
    });  
});
```

Let jQuery AJAX Change This Text

Get External Content

jQuery and AJAX is FUN!

This is some text in a paragraph.

Get External Content



get() and post() method

- ▶ Two commonly used methods for a request-response between a client and server are: GET and POST.
- ▶ **GET** - Requests data from a specified resource
- ▶ **POST** - Submits data to be processed to a specified resource



get()

- ▶ The \$.get() method requests data from the server with an HTTP GET request.

Syntax:

`$.get(URL, callback);`



Example

```
$(document).ready(function(){  
  $("button").click(function(){  
    $.get("demo_test.asp", function(data, status){  
      alert("Data: " + data + "\nStatus: " + status);  
    });  
  });  
});
```

An embedded page on this page says

Data: This is some text from an external ASP file.

Status: success

OK

post()

- ▶ The \$.post() method requests data from the server using an HTTP POST request.

Syntax:

`$.post(URL,data,callback);`



Example

```
$(document).ready(function(){  
  $("button").click(function(){  
    $.post("demo_test_post.asp",  
    {  
      name: "Ahmed",  
      city: "Karachi"  
    },  
    function(data,status){  
      alert("Data: " + data + "\nStatus: " + status);  
    });  
  });  
});
```

An embedded page on this page says

Data: Dear Ahmed. Hope you live well in Karachi.

Status: success

OK